Candidate: **Lakshman** | GitHub: [@Lakshman3556](https://github.com/Lakshman3556)

Position: AI Engineering Internship ? AffineSurge / Tri9T AI

PART 1: The Entire Project Explained in Plain English

1.1 The Real-World Story Behind This Project

Imagine a medical device manufacturer produces a digital blood pressure monitor: the **CardioTrack CT-200**.

Because this is a medical device, strict regulatory standards (such as **FDA 21 CFR Part 820** and **ISO 13485**) require that every single medical device function must have verified **Traceability**:

1. An engineering specification exists in a technical manual (e.g., *"Under cuff overpressure, the emergency valve must auto-deflate within 2.0 seconds"*).
2. A Quality Assurance (QA) engineer writes a specific **Test Case** that points directly to that requirement.

3. Every test case must prove what section and exact words it came from.

1.2 The Core Problem: "Stale Traceability"

Next month, the engineering team releases **Version 2** of the manual. In v2, they changed the deflation threshold from **2.0 seconds** to **1.5 seconds**.

? In traditional software, nobody notifies the QA testing team.

? The test suite continues running the old test case checking for 2.0 seconds.

? The device might fail in hospitals, but the test passes! This is called **Stale Traceability**?one of the most dangerous and common failure modes in medical software.

1.3 What the Examiners Asked Us to Build

The interviewers asked us to build a complete backend API that:

1. Reads the messy technical PDF and reconstructs a clean, hierarchical tree of sections.
2. Supports uploading a new version (v2) of the manual without destroying v1.
3. Lets an engineer select specific sections to generate QA test cases using an AI (LLM).
4. Dynamically warns the engineer when any test case becomes **outdated (stale)** because the underlying requirement changed in a new version.

PART 2: Project Layout & Folder-by-Folder Guide

Here is what every single folder and file in the project does:

ct200-qa-api/

??? app/

? ??? parser/ <-- Step 1: Reads raw PDF and builds a clean outline tree

? ??? services/ <-- Step 2: Ingestion, 5-pass version matcher, diffing, staleness

? ??? llm/ <-- Step 3: LLM client, prompts, and Pydantic validation

? ??? stores/ <-- Step 4: Storage for test cases (JSON file / MongoDB)

? ??? api/ <-- Step 5: FastAPI REST endpoints (browse, search, selections)

? ??? db.py & orm.py <-- SQLite schema with foreign keys

? ??? config.py <-- Environment configuration

??? scripts/ <-- Inspection, naive baseline parser, demo, git commits

??? tests/ <-- 19 automated unit tests

??? docs/ <-- Approach document, debug logs, REST client file

Detailed File Responsibilities:

`app/parser/` (The Extraction Pipeline)

? `extract.py`: The conductor. Opens the PDF, computes the left margin dynamically, coordinates table extraction, classifies lines, and builds the tree.

? `classify.py`: Classifies each line as `TITLE`, `HEADING`, `LIST_ITEM`, `LIST_CONT`, or `BODY` based on bold font, regex, margin position, and table exclusions.

? `tables.py`: Solves PyMuPDF's table extraction bug. Extracts clean cell text by clipping words to each cell bounding box, eliminating comma corruption.

? `tree.py`: Maintains an ongoing hierarchy stack across pages. Resolves parents by longest numeric prefix (e.g. `2.1.1.1` under `2.1`), attaches tables at their vertical `y0` position, and tracks flags (`level_gap`, `out_of_order`, `duplicate_title`).

? `normalize.py`: The text purifier. Uses Unicode NFKC to turn ligatures (`fi`, `fl`, `ff`) into plain ASCII letters, standardizes hyphens, collapses whitespace, and computes SHA-256 hashes (`own_hash` excludes section numbers so renumbering isn't confused with content changes).

? `validate.py`: The invariant watchdog. Asserts multiset word coverage (ensuring 0 words were silently lost), checks for cycles, verifies table counts, and checks that no glued tokens exist.

? `ocr_check.py`: The optional OCR cross-check. Rasterizes pages at 200 DPI and checks similarity against native text using difflib and Jaccard. If tesseract is missing, gracefully skips without crashing.

`app/services/` (Business Logic & Intelligence)

? `ingest.py`: Manages the database transaction when a PDF is uploaded. Fingerprints the file with SHA-256 for idempotency, calls the parser, runs version matching against the prior version, and writes immutable rows.

? `matching.py`: The 5-pass conservative matcher that pairs nodes between v1 and v2:

* Pass 1: Exact key (`section_number` + heading + matched parent)
* Pass 2: Title under matched parent (detects renumbering)
* Pass 3: Unique content hash (detects relocated sections)
* Pass 4: Mutual best fuzzy match (token set ratio >= 90 and body ratio >= 0.6)
* Pass 5: Leftovers (unmatched new $\to$ added, unmatched old $\to$ removed). Never forces a match!

? `diffing.py`: Computes word-level diff opcodes and table row diffs. Features a **lexical significance classifier** that distinguishes *material* changes (digits, medical units like `mmHg`, error codes `E3`, modal words `must`/`within`) from harmless *wording* edits.

? `selection.py`: Manages version-pinned selections. Reconstructs `<source_document>` XML excerpts with ancestor breadcrumbs (e.g. `4 Alarms > 4.2 Error Codes`), and enforces a 12,000-character budget (HTTP 413) to prevent silent prompt truncation.

? `generation.py`: Orchestrates the LLM test case generation. Includes an in-process lock per selection, a deduplication policy (`reused: true`), a 2-step automated repair loop for malformed JSON, and a strict **grounding check** that drops any test case quoting hallucinated text.

? `staleness.py`: Evaluates staleness dynamically at query time against any target version. Checks if evidence quotes are still verbatim substrings of the active section, checks diff significance, and flags cross-reference changes.

`app/llm/`

? `schemas.py`: Pydantic models for medical device test cases (`title`, `steps`, `expected_result`, `evidence_quotes` min 12 chars).

? `prompts.py`: Versioned prompt template (`prompt_version = 'v1'`) that treats manual text as strict data, forbidding invented manufacturer names or safety thresholds.

? `client.py`: Universal client supporting OpenAI-compatible endpoints (Groq, Gemini, OpenRouter) and a built-in deterministic `mock` provider so reviewers can run the project offline with zero keys.

`app/stores/`

? `base.py`: Abstract `GenerationStore` interface.

? `json_store.py`: Default zero-setup JSON file store (`generations.json`).

? `mongo_store.py`: MongoDB backend with automatic JSON fallback if MongoDB is not running.

PART 3: Top 20 Interview Questions & Model Answers

Question 1: "Can you give me a high-level walkthrough of what you built?"

Answer:

"I built a production-grade FastAPI service designed to solve the problem of stale traceability in regulated medical device software.

The architecture consists of four distinct layers:

1. **A deterministic, geometry-aware PDF extraction engine:** It extracts titles, headings, and tables from the CT-200 manual while handling all real-world structural irregularities (like 11pt bold headings and page-break splits) without silent data loss, verified by an invariant multiset word coverage validator.

2. **A relational versioning engine:** It persists immutable document snapshots in SQLite. When version 2 of the manual is uploaded, a conservative 5-pass top-down matcher identifies exactly what was modified, added, or unchanged while preserving original v1 rows.

3. **A grounded LLM generation pipeline:** Users create version-pinned selections that feed into an LLM client with strict Pydantic schemas, an automated 2-step repair loop, and grounding checks that eliminate hallucinated quotes.

4. **Dynamic retrieval-time staleness detection:** When users query test cases, the system dynamically compares the cited evidence quotes and source text against the target document version, categorizing test cases as `fresh`, `likely_stale`, `review_recommended`, or `probably_valid`."

Answer:

"Because `ct200_manual.pdf` is a born-digital PDF. When I inspected the document with PyMuPDF, I verified that it contains exactly zero raster images, embedded vector fonts (`NimbusSans`), and a 100% clean digital text layer.

Running pure OCR on a born-digital document would introduce optical recognition noise?confusing digits like `1` with letters like `l` or dropping commas in medical thresholds like `0.5 mmHg`. In medical device software, that kind of noise is dangerous.

Instead, I used **native-first vector extraction** with PyMuPDF for 100% character fidelity, but integrated an **OCR cross-check module** (`app/parser/ocr_check.py`). It rasterizes pages at 200 DPI, runs Tesseract, and computes difflib similarity and word Jaccard against the native text. Furthermore, if any page contains fewer than 20 characters (a true scanned page), the pipeline automatically falls back to full OCR."

Question 3: "Why does standard font-size thresholding fail on headings in this manual?"

Answer:

"In the CT-200 manual, Section `3.2 Cuff Inflation Sequence` and Section `2.1.1.1 Battery Life Under Typical Use` are rendered in **11.0 pt bold**?which is the exact same font size as the body paragraphs (11.0 pt regular).

If a parser uses a rule like `size > 12.0 pt` to find headings, both `3.2` and `2.1.1.1` are completely missed and demoted into the preceding section's body text.

I solved this by using a **compound classification rule**: a line is classified as a heading if it is bold, matches a section numbering regex (`HEAD_RE`), is aligned at the left margin (`x0 <= margin + 3.0 pt`), and does not intersect a table bounding box. The parser records a `size_mismatch` warning in the report rather than hiding it."

Answer:

"A simple regex check for `^\d+\.` will mistake `1. Normal` for Section 1.

My classifier (`app/parser/classify.py`) checks two additional properties:

1. **Font weight:** Section headings are always bold, whereas list items are regular weight.

2. **Indentation:** Heading lines are anchored at the left margin (`x0 = 33.0 pt`), whereas the list items in Section 3.3 are indented at `x0 = 50.8 pt` (and continuation lines wrap at `x0 = 63.0 pt`).

Because list lines are regular weight and indented, they are classified as `LIST_ITEM` and `LIST_CONT` blocks attached to Section 3.3."

Question 5: "How did you handle the duplicate heading 'Error Codes' appearing in Sections 4.2 and 7.1?"

Answer:

"If you store nodes in a dictionary keyed by title, Section 7.1 will overwrite Section 4.2.

In my system:

1. Every node is assigned a unique UUID upon extraction.

2. The database separates `LogicalNode` from `NodeVersion`.

3. The version matcher matches nodes **top-down** (root first, then children of matched parents). Because Section 4 is matched to Section 4, its child `4.2 Error Codes` can only match with `4.2 Error Codes`. Similarly, `7.1 Error Codes` can only pair with Section 7.1.

4. The parser also detects duplicate titles and flags both nodes with `duplicate_title`."

Question 6: "What is the Word-Multiset Coverage Invariant in Section 5.8?"

Answer:

"The assignment warned that a parser that looks clean but silently loses, merges, or mis-parents content is unacceptable.

To guarantee zero content loss, I built `InvariantValidator` (`app/parser/validate.py`). It extracts all normalized words directly from the source pages and compares them against the multiset of words assembled across all node headings, paragraphs, and list blocks.

If even a single sentence is dropped or lost during parsing, the multiset difference triggers a hard failure, aborting ingestion and listing the exact missing words."

Question 7: "Why did you exclude the section number from `own_hash`?"

Answer:

"`own_hash` is computed from the normalized heading title and the canonical body text: `sha256(heading_title + '\n' + canonical_body)`.

I intentionally excluded the section number (e.g. `'2.1'`). If the author renumbers a section from `2.1` to `2.2` without changing its text or title, the content hash remains identical. This allows the system to classify the change accurately as **'renumbered'** rather than falsely claiming the medical content was modified."

Answer:

"In Version 2, layout reflow caused page breaks to shift (for example, in v2 Section 8.1 spills two lines onto Page 7, whereas in v1 it ended on Page 6).

A naive parser that processes pages in isolation treats page breaks as line joins or introduces false spaces (e.g. `'notuse'` or spurious linebreaks). My parser maintains ongoing state across pages: when a paragraph splits across a page boundary, it joins the text with a single space and flags `split_across_pages`. Because `normalize_text()` collapses all whitespace, layout reflow does not alter the hash, and Sections 3.1, 6.1, and 8.1 are correctly identified as **100% unchanged**."

Question 9: "Explain your 5-pass Version Matcher."

Answer:

"`app/services/matching.py` implements a conservative, one-to-one matcher:

? **Pass 1 (Exact Key):** Same section number, same normalized heading, under parents that have already been matched.

? **Pass 2 (Title Under Parent):** Same heading under matched parents, but section number changed $\to$ identifies renumbering.

? **Pass 3 (Unique Hash Match):** Unmatched nodes with an identical content hash $\to$ identifies moved or relocated sections.

? **Pass 4 (Mutual Best Fuzzy Match):** Uses RapidFuzz token set ratio (≥ 90) and difflib body similarity (≥ 0.6). Candidates must be mutually best to prevent cross-matching.

? **Pass 5 (Leftovers):** Remaining new nodes become `added`; unmatched old nodes become `removed`. We never force a match."

Question 10: "Where does your version matching strategy break? (Known Failure Modes)"

Answer:

"I documented four specific failure modes in `APPROACH.md`:

1. **Heavy Rewrites:** If an author rewrites both the section title and the majority of the body text, the fuzzy thresholds fail. The node will show as 'removed' and 'added' as a new section. Traceability to old test cases is marked `source_removed`. (This is deliberate: a false match is worse than a broken link).

2. **Splits and Merges:** If one section is split into two, or two sections merge into one, the matcher cannot detect the 1-to-many relationship; it classifies one as modified and the other as added/removed.

3. **Simultaneous Renumbering & Retitling:** If a short section is renumbered and retitled at the same time, it drops below the 0.6 body threshold.

4. **Tuned Thresholds:** The thresholds (0.6 body, 90 title) were tuned on this document pair; they may require recalibration on different manuals."

Question 11: "What does 'Version Pinning' mean in your Selection API?"

Answer:

"When a user creates a Selection (e.g. sections `3.2` and `4.2`), the selection does not merely store logical IDs. It pins the exact **`node_version_id`** row and stores the content hash (`pinned_hash`).

Selections are strictly immutable (no `PUT` or `PATCH`). When Version 2 of the manual is uploaded, a historical selection still points to the immutable Version 1 snapshot. Calling `GET /selections/{id}/text` returns the exact text from Version 1 ('40 mmHg'), while `GET /nodes/{id}` returns the active Version 2 text ('30 mmHg')."

Question 12: "How do you distinguish a one-word cosmetic change from a changed pressure threshold?"

Answer:

"I built a lexical significance classifier (`app/services/diffing.py`). It analyzes the difflib opcodes of changed tokens:

? **Material Change:** Any changed token that contains a digit, a medical/physical unit (`mmHg`, `seconds`, `%`, `bpm`, `cycles`), an error code identifier (`E1`..`E6`), or a safety modal (`must`, `shall`, `only`, `within`, `exceeds`, `below`, `above`).

? **Wording Change:** Changes that only involve descriptive prose without numbers or safety boundaries.

In Section 3.2, changing `'40 mmHg'` to `'30 mmHg'` changes a digit and a unit, so it is classified as **material**. If an author fixes a typo or edits grammar without touching numbers, it is classified as **wording** (`probably_valid`)."

Question 13: "What happens if the LLM produces malformed or incomplete output?"

Answer:

"We follow a strict policy defined in Table 9.5 of the brief:

1. **Markdown Fences:** We strip fences (` ```json `) and extract the first balanced `{...}`.

2. **Syntax or Schema Errors:** We initiate an automated 2-step repair retry loop (`LLM_MAX_REPAIR_ATTEMPTS = 2`), sending the original excerpt, the previous raw output, and the exact Pydantic validation error back to the model.

3. **Grounding Check:** Every test case must cite verbatim evidence quotes. If a quote is not a substring of the source excerpt, that test case is dropped. If fewer than 3 valid cases remain, repair is triggered.

4. **All Attempts Fail:** The generation is saved with `status: 'failed'` along with all raw attempts for auditability, and the API returns **HTTP 502 Bad Gateway**. We **never** return partial unvalidated model outputs to QA engineers."

Question 14: "What happens if the same selection is submitted twice to the generation API?"

Answer:

"Our policy is **Idempotent by content, explicit regeneration**.

? The deduplication key is `(selection_fingerprint, prompt_version, model)`.

? If a completed generation already exists with that key, the endpoint immediately returns the existing generation with `reused: true` and does **not** call the LLM.

? We protect this with an in-process `asyncio.Lock` to prevent simultaneous double-click race conditions.

? To regenerate, the caller must explicitly pass `?force=true`, which creates a new generation linked to the old one via `supersedes`.

Why? LLMs are non-deterministic and cost money. Silently generating two different test suites for the exact same input forces QA engineers to reconcile conflicting truths."

Question 15: "Why is Staleness computed at retrieval time instead of stored as a database flag?"

Answer:

"Because staleness is relative to the target version you are comparing against.

A test case generated against Version 1 is `fresh` when compared against Version 1, but `likely_stale` when compared against Version 2.

If staleness were a static boolean column in the database, re-ingesting a new document version would require running bulk write updates across historical records, corrupting historical audit logs. By computing staleness dynamically at retrieval time (`?against_version=latest`), historical records remain immutable and users can query against any version."

Question 16: "What is the hidden requirement-level conflict in Version 2 (Section 2.6)?"

Answer:

"In Version 2, Section `4.1` was unchanged and still states that the emergency valve vents the cuff within **2 seconds**. However, the table in Section `4.2` was updated to state that Error Code `E3` auto-deflates within **1.5 seconds**.

A standard text-diff staleness check on Section 4.1 will report that test cases are `fresh`, because Section 4.1's text did not change! Yet the manual is now internally contradictory.

I highlighted this in `APPROACH.md` as the primary blind spot of lexical staleness detection, and built a diagnostic hint: when a node changes, the API lists `possibly_related_nodes` that share the same error codes (`E3`) or units (`seconds`)."

Answer:

"The staleness detection engine. It evaluates whether the specific text cited by a test case still exists in the latest document version. It cannot see cross-section semantic contradictions?such as Section 4.1 remaining unchanged at 2 seconds while Section 4.2 changes to 1.5 seconds.

I caught this by adding a related-nodes consistency hint (linking shared identifiers like `E3` and units like `seconds`), human review for material diffs, and explicit unit tests documenting the gap."

Answer:

"I chose a heuristic lexical significance classifier (checking digits, units, and modal words) rather than building a semantic AST parser for medical requirement logic.

What would break first in production is subtle phrasing reversals that invert a requirement without using trigger words (e.g. descriptive medical prose replacing 'must'). Furthermore, in high-throughput production, the in-process lock and SQLite database would need to be migrated to PostgreSQL and a distributed Redis/Celery queue."

Answer:

"For the PDF parser, I did not handle multi-column layouts or complex rotated graphical schematics. If a multi-column PDF is uploaded, the parser detects that text bounding boxes cross column gutters, marks the document with `parse_status = 'needs_review'`, and emits an `unsupported_block` warning rather than silently scrambling the reading order.

For the LLM endpoint, if a selection's text exceeds 12,000 characters, rather than silently truncating it, the API rejects the request with **HTTP 413 Payload Too Large**."

Answer:

? **To add a new modal word to the significance classifier:** Open `app/services/diffing.py` $\to$ edit the `MODALS` set (Line 14).

? **To change the fuzzy matching threshold:** Open `app/services/matching.py` $\to$ adjust `heading_sim >= 90` or `body_sim >= 0.6` (Lines 118?121).

? **To adjust the duplicate generation policy:** Open `app/services/generation.py` $\to$ edit deduplication logic in `generate_test_cases()` (Lines 60?75).

? **To modify the selection character budget:** Open `app/config.py` $\to$ edit `MAX_SELECTION_CHARS` (Line 18).